

SpMV CUDA Kernel

```
// CSR SpMV (base.cu:22)
for (int offset = row_begin;
    offset < row_end; offset++) {
    sum += Ax[offset]*x[Aj[offset]];
}
```

Data Load Index Load

Target Table(TT)

Idx	Valid	Base Address	Scale	Type
0	Y	0x7FC4B7C20000	4B	f32
1	Y	0x7FC4B7E40000	4B	f32
2	N	0x7FC4B8100000	8B	f64
...	...	...	...	...

$$Addr_{target} = Addr_{base} + scale \times index$$

Chain Table(CT)

Identification		Target Mapping			Stride Learning(IST)					
Valid	Index PC	TT Index			K	Stride Obs		Iter Stride	Stride Valid	
		[0]	[1]	[2]		[0]	[1]			
Y	0x03d0	0	—	—	1	W5: 0x7FC4D0DA4864	W5: 0x7FC4D0DA48A4	64B	Y	
Y	0xA120	0	1	—	2	W3: 0x7FC4D0DA2100	W3: 0x7FC4D0DA2110	16B	Y	
N	0xB340	0	1	2	3	W8: 0x7FC4D0DA5040	—	4B	N	
...	...	...	...	...	...	...	...	...	...	

SASS Assembly(x16 Unrolled Loop)

```
1 /*0x03d0*/ LDG.E R9,[R6+-0x8]
2 /*0x03e0*/ LDG.E R27,[R6+-0x4]
3 /*0x03f0*/ LDG.E R13,[R6]
4 ... (13 more index loads)
5 /*0x0480*/ IMAD.WIDE R8,R9,R16,c[0x180]
6 /*0x0490*/ IMAD.WIDE R26,R27,R16,c[0x180]
7 /*0x04b0*/ IMAD.WIDE R12,R13,R16,c[0x180]
8 ... (13 more IMAD.WIDEs)
9 /*0x04a0*/ LDG.E R14,[R8]
10 /*0x04c0*/ LDG.E R24,[R26]
11 /*0x04D0*/ LDG.E R12,[R12]
12 ... (13 more data loads)
13 /*0x0580*/ FFMA R14,R14,R18,R11
14 ... (accumulation)
```

IMA Index Load

Addr Generation

IMA Data Load

Zoom:

How Iter Stride is Computed

W0 @ Iter i: addr=0x7FC4D0DA4864

W0 @ Iter i+1: addr=0x7FC4D0DA48A4

Stride = 0x48A4 - 0x4864 = 0x40 = 64 B